

Requirements, Design and Test Documentation

Dining Philosophers für VSP

Hert, Tom

Tom.Hert@haw-hamburg.de

Zacharias, Laurin

Laurin.Zacharias@haw-hamburg.de

30. Januar 2025

Inhaltsverzeichnis

1	Aufgabenstellung	3
2	Anforderungen	4
3	Entwurf	5
4	Remote Procedure Calls	6
5	RPC Command Struktur	7
5.1	Response	7
5.2	Commands	7
6	Logik	9
7	Frontend	10
8	Analyse	12
8.1	Performance	12
8.2	Effizienz	12
8.2.1	Gerade Anzahl an Philosophen	12
8.2.2	Ungerade Anzahl an Philosophen	13

1 Aufgabenstellung

Sie setzen eine Lösung des bekannten Philosophenproblems um. Einzelne Teilnehmer, hier Philosophen, werden in einzelnen Container realisiert. Diese kommunizieren ausschließlich mittels Nachrichten und die Zugriffe der Teilnehmer werden verteilt koordiniert, d.h. keine (pseudo-) zentrale Lösung. Es sollen Performance-Messungen für 5, 10 und eine von Ihnen zu ermittelnde maximal Anzahl von Container realisiert werden. Die RPC Lösung ist hierbei komplett selbst zu erstellen und durch Tests abzusichern, bestehende Lösungen wie `grpc`, `rmi`, ... sind nicht zugelassen.

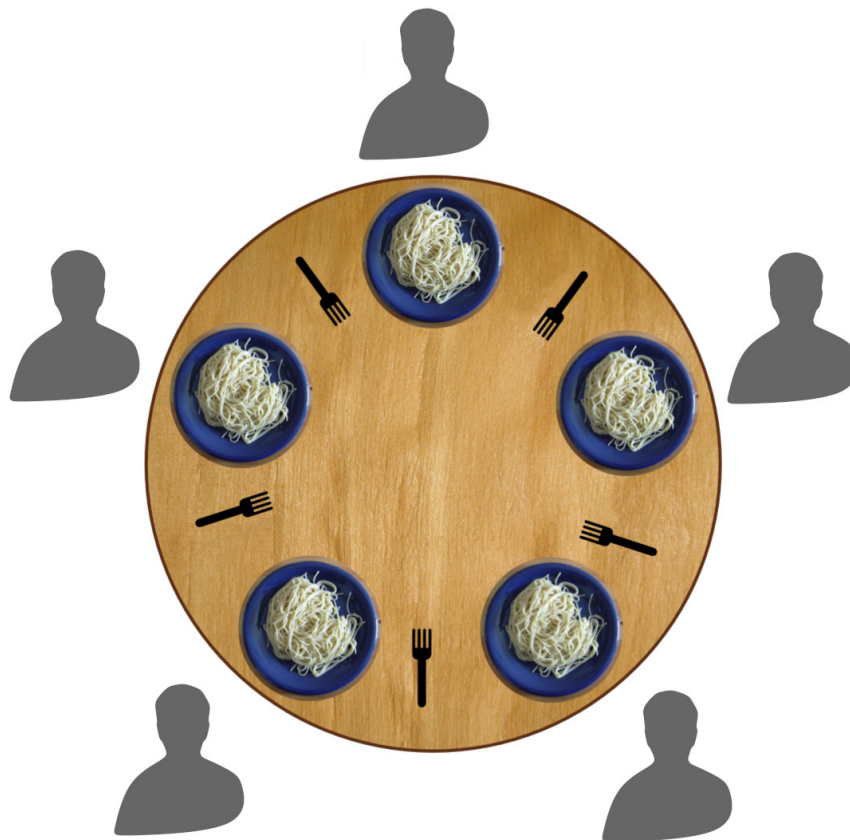


Abbildung 1: Dining Philosophers

2 Anforderungen

1. Besteck kann nur von einem Philosophen gleichzeitig benutzt werden.
2. Es gibt genauso viele Bestecke wie Philosophen.
3. Philosophen müssen einzelne Container sein.
4. Philosophen werden können entweder Essen oder Denken:
 - (a) wenn sie Denken interagieren sie nicht mit anderen Philosophen oder Besteck.
 - (b) wenn sie Essen wollen benötigen sie die 2 angrenzenden Bestecke um diese Tätigkeit zu vollführen und legen diese anschließend wieder ab.
5. Philosophen sollen ausschließlich die 2 angrenzenden Bestecke benutzen können.
6. Philosophen können nicht wissen wann die anderen Philosophen Essen oder Denken wollen.
7. Philosophen müssen im Wechsel Essen und Denken können sodass sie nicht verhungern.
8. Philosophen verhungern wenn sie nicht Essen können.
9. Philosophen dürfen ausschließlich per Nachrichten kommunizieren und ihre Zugriffe sollen verteilt koordiniert werden.
10. Die Performancemessungen sollen mit 5, 10 sowie einer zu ermittelnden maximal Anzahl stattfinden.
11. Zur verteilten Koordination soll eine eigene Lösung für RPC genutzt werden und keine Bibliothek.
12. Die eigene RPC Lösung muss mit Tests abgesichert werden.

3 Entwurf

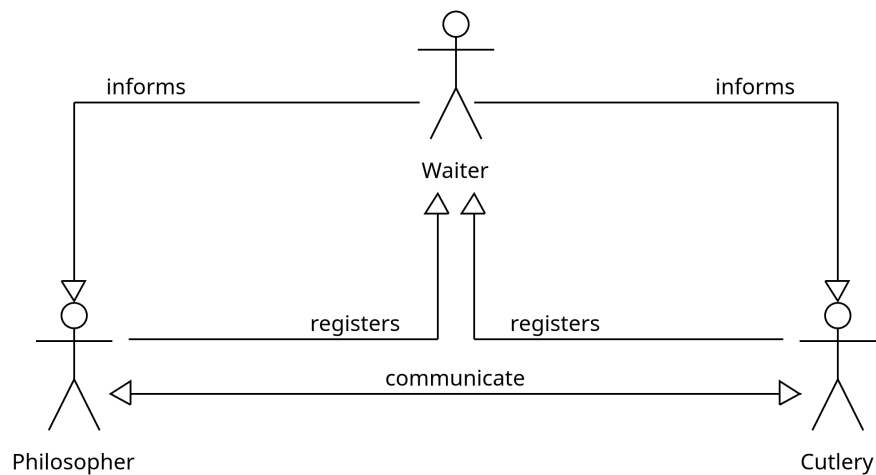


Abbildung 2: Das Setup der Container

Zur Kommunikation zwischen den verschiedenen Containern nutzen wir ein zentrales Melderegister namens Waiter. In diesem müssen sich Cutlery und Philosopher registrieren, um miteinander kommunizieren zu können. Der Waiter ist ein zentraler Bestandteil des Systems und wird von allen Containern benötigt, um sich gegenseitig zu finden. Die Kommunikation zwischen den Containern erfolgt über das Publish-Subscribe-Prinzip, d.h. dass man nach der Registrierung beim Waiter über seine angrenzenden registrierten Container informiert wird.

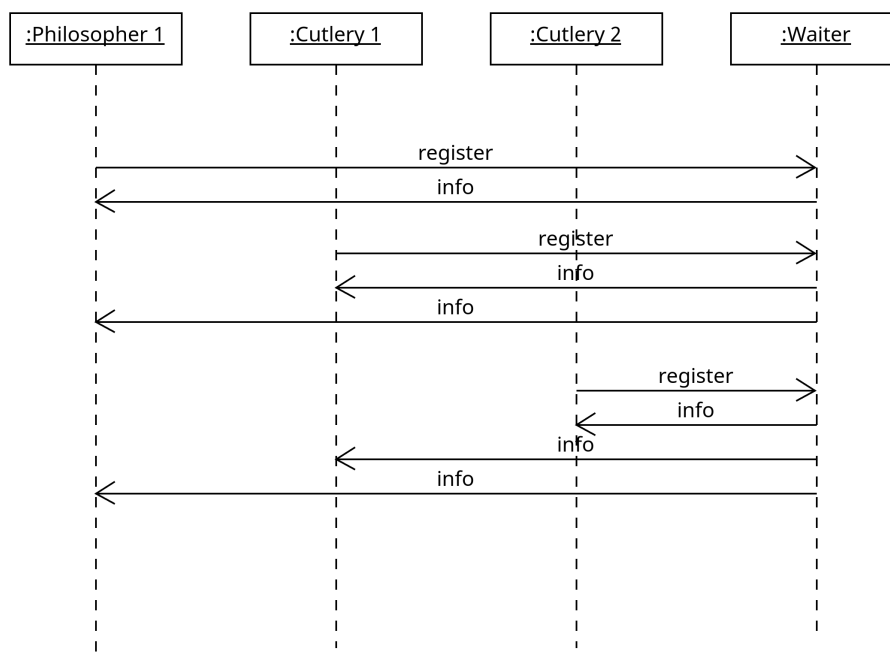


Abbildung 3: Das Publish Subscribe Prinzip vom Waiter

4 Remote Procedure Calls

Die RPCs sind mit einem Trait 'Calls' zur Bestimmung der Signaturen aller remote rufbaren Funktionen der Container deklariert. Zur Kommunikation nutzen wir TCP Sockets über die wie folgend Nachrichten zum Funktionsaufruf als bytes codiert gesendet werden:

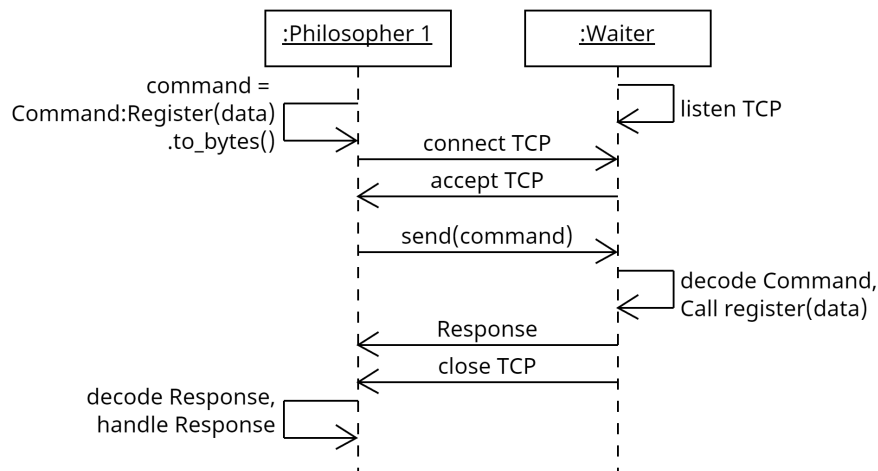


Abbildung 4: RPC Ablauf

- **Commands** ist ein enum das alle möglichen Funktionsaufrufe an Nodes umfasst und beim Senden eines RPCs in bytes codiert und gesendet wird und anschließend vom Empfänger ins enum decodiert und als Funktionsaufruf ausgeführt wird.
- Das Antwortformat auf einen RPC ist ein enum **Response**: (Success, Failure(String), Return(Vec<u8>), NotFound), das je nach Ausgang des Aufrufs Daten zurückliefern kann.

Abstrahiert sieht das ganze für einen Philosophen oder Waiter dann so aus:

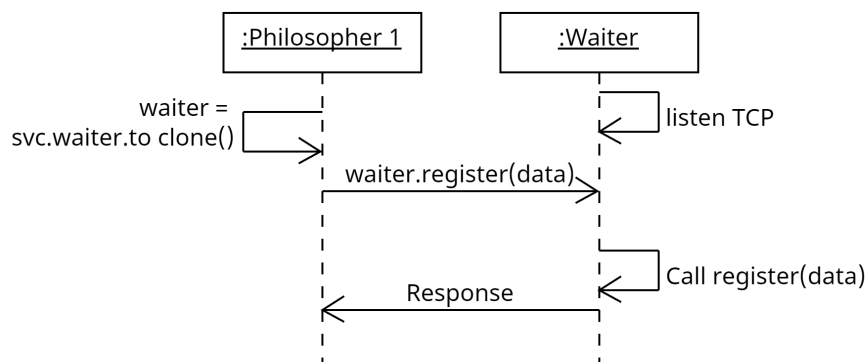


Abbildung 5: RPC Abstrahiert

5 RPC Command Struktur

Die RPC Command Struktur definiert die verschiedenen Befehle und Antworten, die zwischen den Knoten eines Netzwerks ausgetauscht werden können. Diese Struktur ist entscheidend für die Kommunikation und Koordination innerhalb des Netzwerks. Im Folgenden werden die verschiedenen Komponenten dieser Struktur beschrieben.

5.1 Response

Die Response Enum repräsentiert die möglichen Antworten auf einen RPC-Aufruf. Diese Antworten können wie folgt aussehen:

- `Success`: Der Aufruf war erfolgreich.
- `Failure(String)`: Der Aufruf ist fehlgeschlagen und liefert eine Fehlermeldung.
- `Return(Vec<u8>)`: Der Aufruf hat Daten zurückgegeben, die als Byte-Vektor serialisiert sind.
- `NotFound`: Der angeforderte Knoten wurde nicht gefunden.

5.2 Commands

Die Commands Enum definiert die verschiedenen Befehle, die an einen Knoten gesendet werden können. Diese Befehle werden als Aufrufe an den Knoten behandelt und von der Implementierung des `Calls Traits` des Knotens verarbeitet. Die Befehle umfassen:

- `Register(Vec<u8>)`: Registriert Knoten im Netzwerk, z.B. einen Kellner.
- `Info`: Fordert Informationen über einen Knoten an, z.B. einen Kellner.
- `Initialize((Node, Node, Option<Node>, Option<Node>), usize)`: Initialisiert Knoten mit bestimmten Parametern.
- `CleanCutlery(Node)`: Reinigt Besteck.
- `UseCutlery(Node)`: Macht Besteck durch Benutzung schmutzig.
- `IsDirty`: Überprüft, ob das Besteck schmutzig ist.
- `ReceiveCutlery(Node, String)`: Ein Philosoph erhält ein Stück Besteck von einem anderen Philosophen.
- `ReceiveRequest(Node, String)`: Ein Philosoph erhält eine Anfrage für ein Stück Besteck, das er hält.
- `PickUp(Node)`: Hebt ein Stück Besteck auf.
- `PutDown`: Legt ein Stück Besteck ab.
- `SetState(States)`: Setzt den Zustand eines Knotens.

- `GetState`: Fordert den aktuellen Zustand eines Knotens an.
- `InformStateUpdate(Vec<u8>)`: Informiert über den Zustand des Knotens, einschließlich vollständiger Knotendaten.
- `ReportStateTime(States, u64)`: Meldet die Zeit, in der ein Zustand aktiv war.

6 Tests

6.1 Testumgebung

Wir verwenden eine Mock-Implementierung des `Calls Traits`, um die verschiedenen RPC-Kommandos zu testen. Diese Mock-Implementierung simuliert die Antworten auf die RPC-Kommandos, ohne dass ein echter Netzwerkverkehr erforderlich ist.

Für einige Tests simulieren wir jedoch auch den echten Netzwerkverkehr, indem wir eine TCP-Verbindung zwischen zwei Knoten herstellen und die RPC-Kommandos über diese Verbindung senden.

6.2 Beispieltests

- `test_get_state`: Testet den `GetState`-Befehl und überprüft, ob die Antwort `Success` ist.
- `test_set_state`: Testet den `SetState`-Befehl und überprüft, ob die Antwort `Success` ist.
- `test_handle_request`: Testet die Verarbeitung eines `GetState`-Befehls und überprüft, ob die Antwort `Success` ist.
- `test_connection_handler`: Testet den Verbindungs-Handler, indem eine TCP-Verbindung hergestellt und ein `GetState`-Befehl gesendet wird. Überprüft, ob die Antwort `Success` ist.
- `test_send_command_to`: Testet das Senden eines `GetState`-Befehls über eine TCP-Verbindung und überprüft, ob die Antwort `Success` ist.

7 Logik

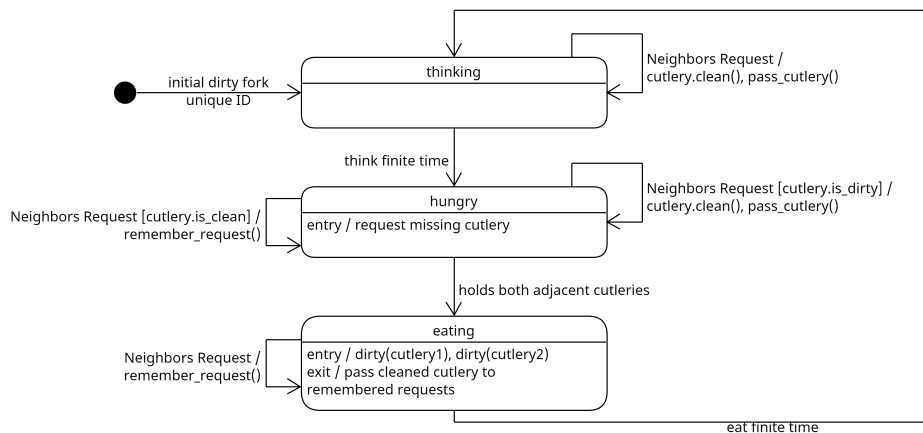


Abbildung 6: Philosoph Logikaufbau

IDs werden von in der Reihenfolge der Registrierung beim Waiter von 0 bis zur Anzahl Philosophen - 1 vergeben.

Besteck wird immer zwischen zwei Philosophen geteilt und darf nur von einem Philosophen gleichzeitig genutzt werden. Es gibt genauso viel Besteck wie Philosophen.

Besteck wird vor dem Abgeben an Nachbarn von Philosophen gereinigt und müssen zu jedem Zeitpunkt einem der zwei angrenzenden Philosophen gehören.

Philosophen haben 3 States:

- **thinking:** der Philosoph denkt eine endliche Zeit, währenddessen gibt er sein Besteck auf Nachfrage an seine Nachbarn ab.
- **hungry:** der Philosoph fragt seine Nachbarn nach dem ihm fehlendem Besteck, gibt nur dreckiges Besteck auf Nachfrage ab und merkt sich Anfragen zu sauberem Besteck.
- **eating:** der Philosoph hat beide Bestecke und isst eine endliche Zeit, währenddessen merkt er sich Anfragen zu Besteck. Das Besteck wird dreckig und vor dem Wechsel zurück zu thinking gibt der Philosoph sein Besteck ab wenn jeweils eine gemerkte Anfrage vorliegt.

Philosophen können Bestecke nicht weggenommen werden.

Initial bekommen Philosophen mit gerader ID keine Bestecke und mit ungerader ID beide Bestecke um eine nicht zyklische Startverteilung zu haben, bei einer ungeraden Zahl Bestecke bekommt der Philosoph mit der höchsten ID ein Besteck.

Wenn die Startverteilung nicht zyklisch ist kann mindestens ein Philosoph zu Beginn den Zustand eating erreichen, dies bedeutet das er anschließend beide Bestecke abgibt sobald seine Nachbarn diese anfragen, die Nachbarn fragen diese erst an wenn sie im Zustand hungry sind und geben in diesen Zustand das Besteck erst wieder ab nachdem sie den Zustand eating erreicht haben, da sie das Besteck sauber erhalten. Damit ist sicher gestellt das keine zyklische Abhängigkeit entstehen kann, weil der Philosoph, der im Zustand eating war, keine Bestecke erhalten kann bis seinen Nachbarn auch den Zustand eating erreicht haben und aufgrund der gleichen Anzahl von Philosophen und Bestecken ein anderer Philosoph zwei Bestecke haben muss.

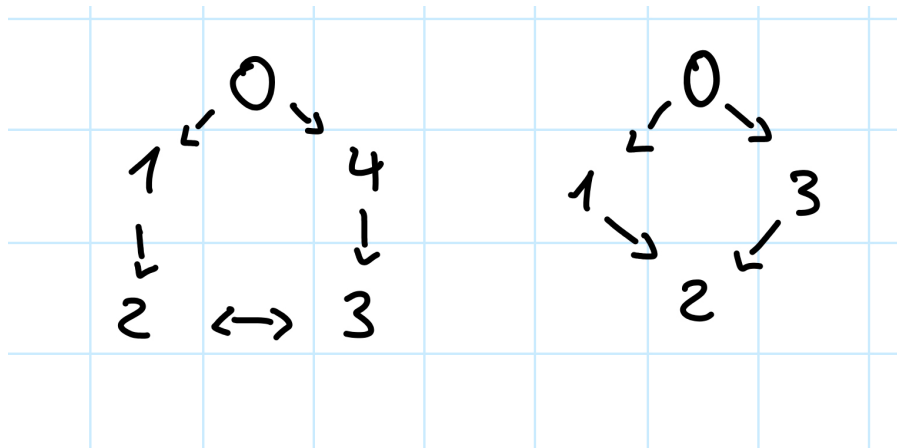


Abbildung 7: Beispiel Verhinderung zyklischer Abhängigkeit

Hier ein Beispiel dazu: 0 hat gegessen, die Pfeile zeigen die Verteilung der Bestecke was darin resultiert das sowohl bei gerader als auch ungerader Anzahl Philosoph einer zwei Bestecke haben muss.

8 Frontend

Für die Visualisierung haben wir ein Frontend in Vite + Typescript geschrieben.

Der Waiter hört auf Port "3001 auf Anfragen und gibt Informationen über die Aktivitäten der Philosophen und Bestecke zurück.

Außerdem sammelt der Waiter Statistikdaten und gibt diese zurück.

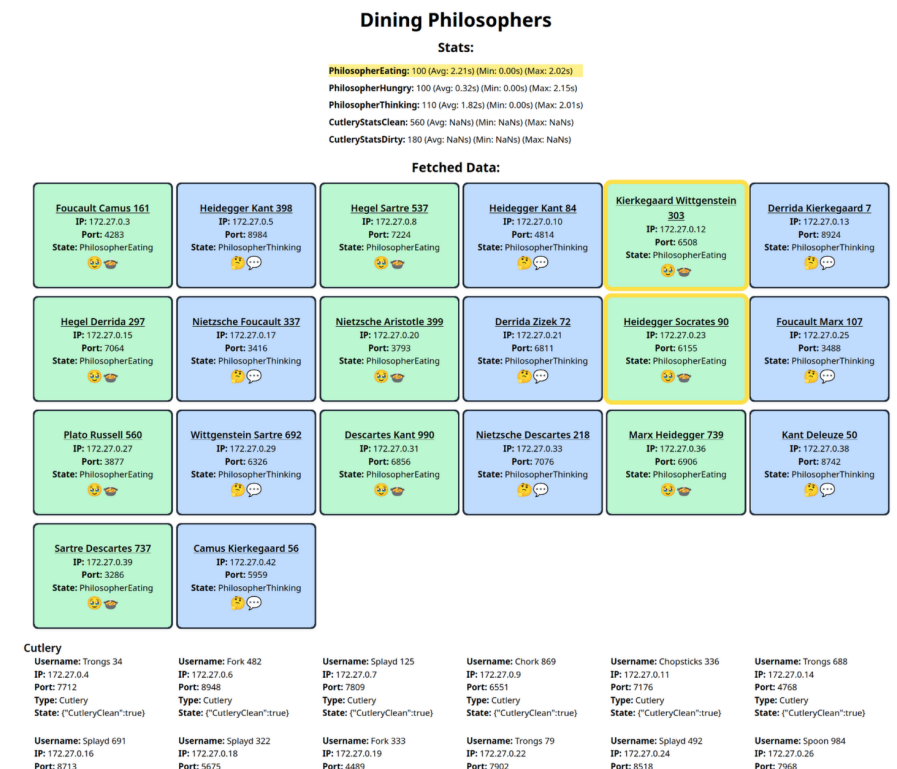


Abbildung 8: Frontend

Das Frontend zeigt durch diese Daten ebenfalls Statusänderungen signalisiert durch Farbänderungen der Philosophen und Bestecke.

9 Analyse

9.1 Performance

Der Durchschnitt bei 50 Philosophen:

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O
dcd773919aad	vsp-cutlery-3	0.00%	1.641MiB / 30.51GiB	0.01%	145kB / 127kB
898fefaf009b1	waiter	0.00%	2.789MiB / 30.51GiB	0.01%	4.42MB / 4.33MB
faff8eb1e008	vsp-philosopher-19	0.00%	1.824MiB / 30.51GiB	0.01%	206kB / 188kB

9.2 Effizienz

Feste Konstanten:

- Zeit Thinking: 2.00s
- Zeit Eating: 2.00s
- Zeit Sleeping pro Versuch: 0.01s

9.2.1 Gerade Anzahl an Philosophen

Anzahl	Eating Avg	Eating Max	Hungry Avg	Hungry Max	Thinking Avg	Thinking Max
4	2.01	2.04	0.13	2.14	1.99	2.00
10	2.02	2.04	0.14	2.15	1.98	2.00
30	2.03	2.03	0.17	2.25	1.98	2.03
50	2.02	2.04	0.17	2.25	1.99	2.04
100	2.09	2.11	0.21	2.36	1.96	2.08

Tabelle 1: Performance Daten (Gerade Zahlen)

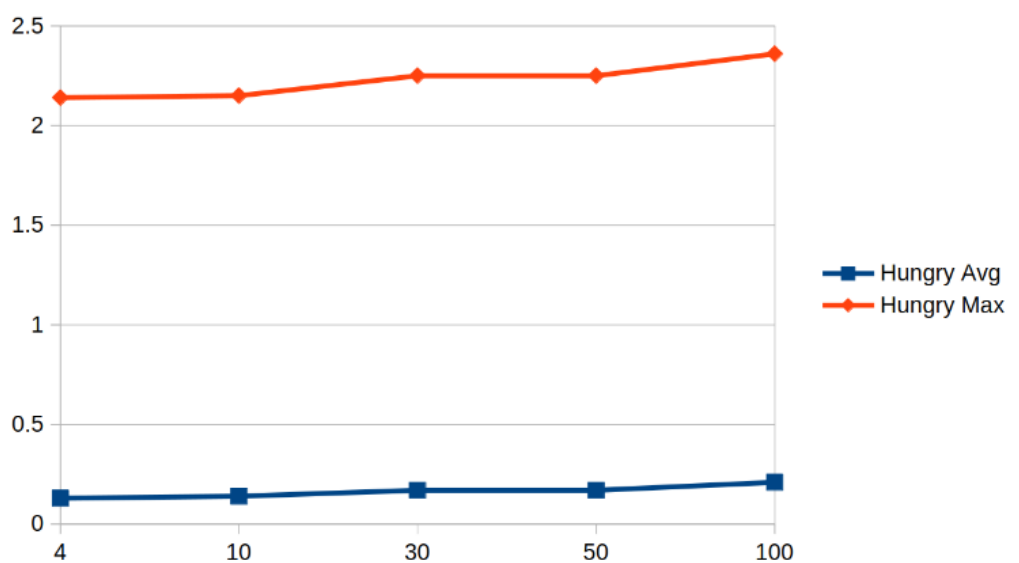


Abbildung 9: Hungry Time bei gerader Anzahl an Philosophen

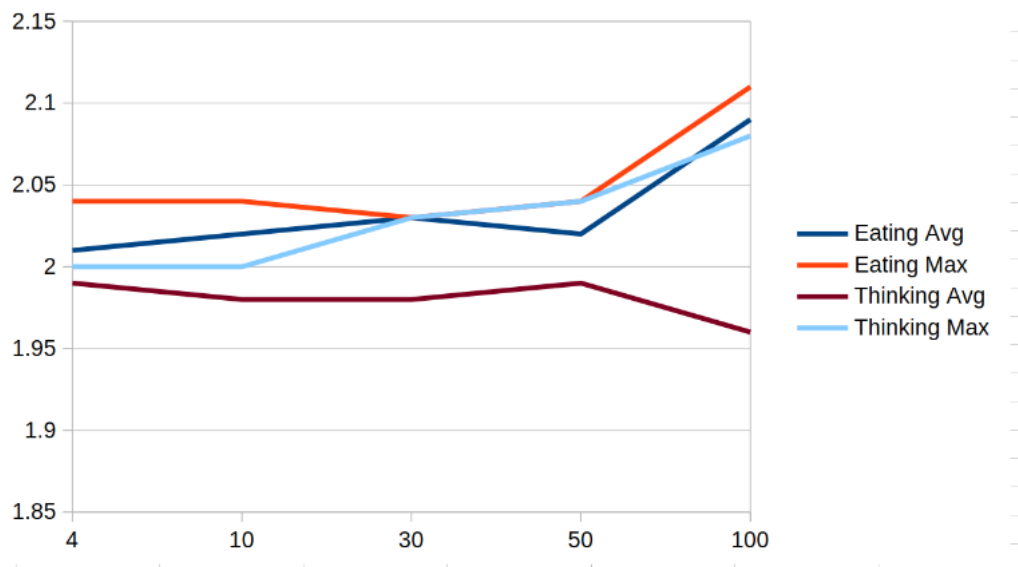


Abbildung 10: Statistiken bei gerader Anzahl an Philosophen

9.2.2 Ungerade Anzahl an Philosophen

Anzahl	Eating Avg	Eating Max	Hungry Avg	Hungry Max	Thinking Avg	Thinking Max
5	2.01	2.04	1.18	4.17	1.98	2.00
15	2.02	2.04	0.43	4.19	1.98	2.00
31	2.02	2.05	0.30	4.18	1.97	2.02
51	2.04	2.06	0.26	4.29	1.97	2.04
101	2.06	2.07	0.22	4.40	1.97	2.07

Tabelle 2: Performance Daten (Ungerade Zahlen)

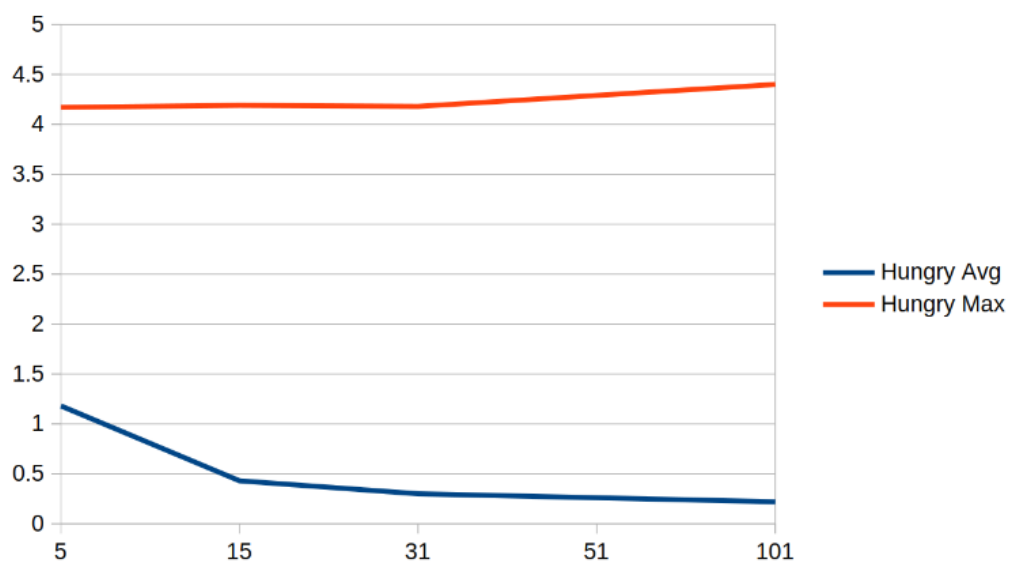


Abbildung 11: Hungry Time bei ungerader Anzahl an Philosophen

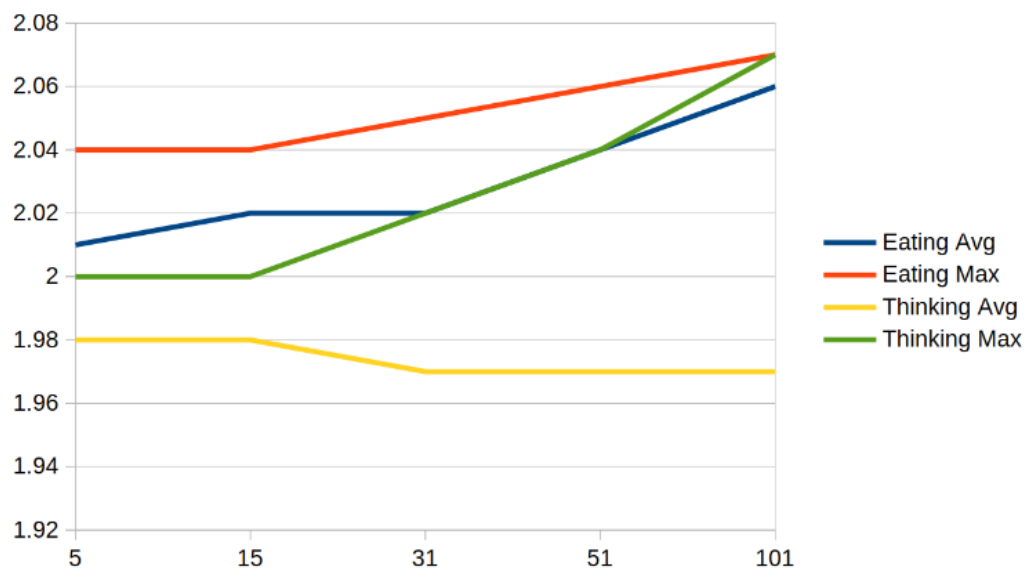


Abbildung 12: Statistiken bei ungerader Anzahl an Philosophen